M E I T Y A I C Y B E R P R O J E C T

Objective 1: Standardisation of Cybersecurity Audit Report Format

TECHNICAL MANUAL

AI-Assisted Cybersecurity Audit Report Standardisation System

1. Introduction

1.1 Purpose of This Manual

This Technical Manual documents the complete system architecture, software and hardware requirements, installation procedures, module-level descriptions, and implementation details of the AI-Assisted Cybersecurity Audit Report Standardisation System developed under Objective 1 of the MEITY AI Cyber Project. It is intended for system developers, administrators, and technical reviewers who need to understand, maintain, deploy, or extend the system.

1.2 What the System Does — Technical Summary

The system is an offline, air-gap-compatible document-intelligence pipeline. It accepts cybersecurity audit reports in heterogeneous formats (PDF, DOCX, TXT, XLSX, scanned images, logs) and transforms them into a uniform, machine-readable, source-traceable JSON output conforming to a standard schema. The pipeline is a sequential, modular architecture of 13 stages. Each stage is implemented as an independent Python module with well-defined input/output contracts, allowing individual components to be updated, replaced, or tested in isolation.

1.3 Novel Models Used

The AI-Assisted Cybersecurity Audit Report Standardisation System incorporates a combination of modern natural-language processing, semantic embedding, large-language modelling, few-shot classification, and optical character recognition technologies to process heterogeneous cybersecurity audit documents. ModernBERT is used as the primary transformer-based model for section detection and named-entity recognition, enabling the system to identify relevant report sections and extract important cybersecurity entities from audit content. BGE-M3 provides semantic embedding capabilities and acts as a fallback mechanism for section matching when the confidence of the primary ModernBERT-based classification is insufficient. Mistral-7B-Instruct-v0.3, deployed using a 4-bit GPTQ configuration, is used for structured information extraction and summary generation from unstructured audit text while supporting the system's offline processing requirement without dependence on external APIs. SetFit is incorporated as a few-shot text-classification approach for severity normalisation, making it suitable for cybersecurity environments where large quantities of labelled severity data may not be available. In addition to these language models, Tesseract OCR v5 combined with OpenCV image preprocessing is used to process scanned or image-based audit documents. The OCR component applies preprocessing operations such as deskewing, adaptive thresholding, and morphological denoising to improve text recognition quality. Together, these models form the core AI and document-intelligence layer of the system, allowing the pipeline to process both digitally generated and scanned audit reports and transform their contents into structured, standardised information

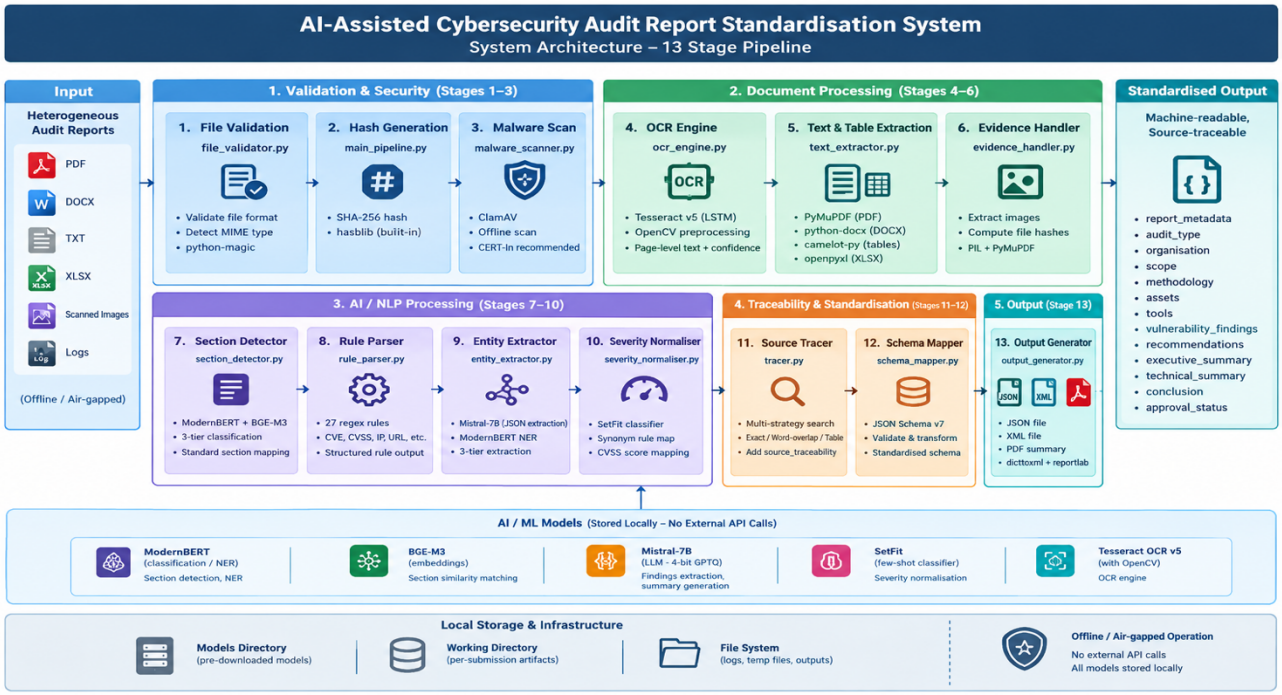
Model	Type	Why Novel / Why Chosen	Used In
ModernBERT (answerdotai/ModernBERT-base, Dec 2024)	Transformer classifier / NER	Latest BERT-variant: 8192-token context, rotary positional embeddings, alternating local+global attention. Outperforms BERT/RoBERTa on classification at same parameter count.	Section detection, NER entity extraction

BGE-M3 (BAAI/bge-m3)	Dense embedding model	State-of-the-art multi-lingual retrieval embedding. Used for semantic similarity when ModernBERT confidence is low.	Section heading similarity matching
Mistral-7B-Instruct-v0.3 (4-bit GPTQ)	Instruction-following LLM	Fully offline, open-weights, runs on 8GB VRAM or CPU. Used for structured JSON extraction from unstructured text — no external API.	Findings extraction, summary generation
SetFit (HuggingFace)	Few-shot text classifier	Achieves BERT-level accuracy with only 8 labelled examples per class using contrastive fine-tuning. Ideal for severity normalisation where cyber-labelled data is scarce.	Severity label normalisation
Tesseract OCR v5 + OpenCV	OCR + image preprocessing	Tesseract v5 uses LSTM-based recognition. OpenCV preprocessing (adaptive thresholding, deskew, morphological denoising) significantly improves accuracy on scanned audit documents.	OCR engine

2. System Architecture

2.1 High-Level Architecture

The system follows a linear pipeline architecture with 13 sequential stages. Each stage is a discrete Python module with a single responsibility. Stages communicate via structured Python dictionaries (passed in memory) and file system artifacts (written to a per-submission working directory). The architecture is designed for offline / air-gapped operation. No external API calls are made during processing. All AI models are downloaded once and stored locally in the models/ directory.

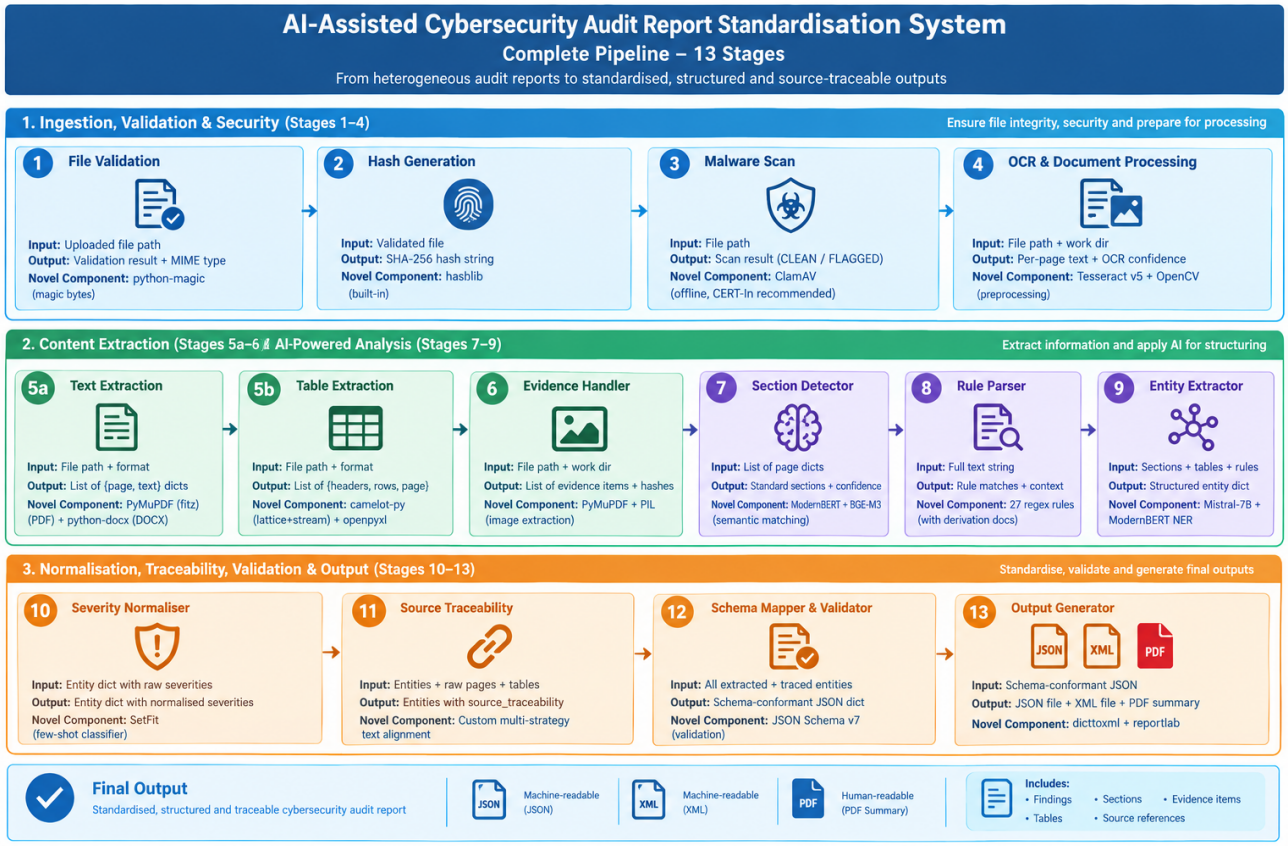


2.2 Complete Pipeline — All 13 Stages

Stage	Input	Output	Novel Component
1	Uploaded file path	Validation result + detected MIME type	python-magic (magic bytes — more reliable than extension)
2	Validated file	SHA-256 hex hash string	hashlib (built-in, deterministic)
3	File path	Scan result: CLEAN / FLAGGED	ClamAV (offline, CERT-In recommended)
4	File path + work dir	Per-page text dict + OCR confidence	Tesseract v5 LSTM + OpenCV preprocessing
5a	File path + format	List of {page, text} dicts	PyMuPDF (fitz) for native PDF; python-docx for DOCX
5b	File path + format	List of {headers, rows, page} dicts	camelot-py (lattice+stream modes) + openpyxl
6	File path + work dir	List of evidence item dicts with file hashes	PyMuPDF image extraction + PIL
7	List of page dicts	List of {standard_section, heading, pages, text, confidence}	ModernBERT + BGE-M3 (novel)
8	Full text string	Dict of rule_name → list of {value, page, context}	27 compiled regex rules with derivation docs
9	Sections + tables + rules	Structured entity dict (findings, tools, orgs, etc.)	Mistral-7B-Instruct + ModernBERT NER (novel)
10	Entity dict with raw severities	Entity dict with normalised severities	SetFit few-shot classifier (novel)
11	Entities + raw pages + tables	Entities with source_traceability fields added	Custom multi-strategy text alignment
12	All extracted + traced entities	Schema-conformant JSON dict	JSON Schema v7 validation
13	Schema-conformant JSON	JSON file + XML file + PDF summary	dicttoxml + reportlab

The complete processing pipeline consists of thirteen sequential stages that transform an uploaded cybersecurity audit report into a standardised, structured, and source-traceable output. The pipeline begins with file validation, where the uploaded document is checked for existence, supported format, size, and content consistency. The validated file is then assigned a deterministic SHA-256 hash and subjected to malware scanning to establish basic security and integrity before further processing. The document-processing stage subsequently determines whether OCR is required and extracts text from

native or scanned documents using Tesseract OCR and OpenCV preprocessing where necessary. Native textual content and structured tables are extracted separately so that both narrative information and tabular findings can be preserved. Embedded screenshots, images, and other supporting evidence are also extracted and associated with the source document.



Following document processing, the extracted content enters the AI and information-extraction layer. The section detection component identifies standard cybersecurity audit sections using ModernBERT and BGE-M3-based semantic matching. A rule-based parser then identifies structured information using predefined patterns, while the entity extraction stage combines Mistral-7B-Instruct with ModernBERT-based named-entity recognition to identify findings, organisations, tools, vulnerabilities, and other relevant entities. The extracted severity values are subsequently normalised into standard categories using the SetFit few-shot classifier. The pipeline then performs source traceability by linking extracted information back to its original page, paragraph, table, or relevant text evidence. This ensures that the structured information remains verifiable against the source audit report.

In the final stages, the extracted and traceable information is mapped into the standard cybersecurity audit schema and validated against the defined JSON schema structure. The output generation stage then converts the schema-conformant information into the required machine-readable and human-readable formats, including JSON, XML, and PDF summary outputs. The sequential design of these thirteen stages ensures that each processing operation has a defined input and output, allowing the overall system to maintain modularity, traceability, validation, and consistency throughout the transformation of heterogeneous audit reports into a unified standardised representation. The pipeline therefore combines conventional document-processing and rule-based techniques with modern AI models to provide a controlled end-to-end cybersecurity audit report standardisation workflow.

2.3 Full Directory / File Structure

The project directory structure provides a modular organisation of the AI-Assisted Cybersecurity Audit Report Standardisation System, with each directory dedicated to a specific functional responsibility within the overall pipeline. The pipeline directory contains the core orchestration and processing modules responsible for validation, malware scanning, text and table extraction, evidence handling, schema mapping, and output generation, while the ocr and parser directories provide specialised components for OCR processing and rule-based information extraction. The ai_module directory contains the machine-learning components responsible for section detection, entity extraction, synonym mapping, and severity normalisation, whereas the traceability directory manages the association of extracted information with its original source locations. The schemas directory maintains the master cybersecurity audit schema together with domain-specific extensions for VAPT, network, web application, and IoT audits. Locally stored AI model weights are maintained in the models directory, while system-wide processing parameters are defined in the config directory. Supporting functionality such as structured logging is maintained under utils, and the tests directory contains automated test modules and sample audit reports used for validation. The output_samples directory provides example structured outputs, while the optional api directory contains the REST API wrapper for external application integration. Finally, the requirements.txt file defines the Python dependencies required by the project. This modular directory structure separates processing logic, AI models, schemas, configuration, testing, and supporting resources, making the system easier to maintain, configure, test, and extend while preserving a clear relationship between the individual components and the thirteen-stage processing pipeline.

2.4 Component Interaction Diagram

The following table shows how data flows between modules — what each module receives, what it produces, and which downstream module consumes that output. The Component Interaction Diagram explains how the individual modules of the cybersecurity audit standardisation system communicate with one another and how information is progressively transformed as it moves through the pipeline. It shows the complete data flow beginning with the user or upload interface, which provides the audit file and associated metadata to the master pipeline. The master pipeline coordinates validation, hashing, malware scanning, OCR, text extraction, table extraction, and evidence extraction, with each module producing a specific data structure that is consumed by one or more downstream components. The extracted text, tables, and sections are then passed to the rule parser and AI-based modules for identifying structured information, entities, and cybersecurity findings. The severity normalisation component standardises the extracted severity values, while the tracer adds source-traceability information that connects each extracted field back to its original document location. Finally, the schema mapper combines the processed information into a schema-conformant structured representation, which is passed to the output generator to produce JSON, XML, PDF, and packaged outputs for the user. Thus, the diagram conveys the producer–consumer relationship between modules, showing not only the sequence of processing but also what data is generated at each stage, where that data is consumed, and how raw audit-report content is progressively transformed into a validated, structured, and traceable cybersecurity audit report.

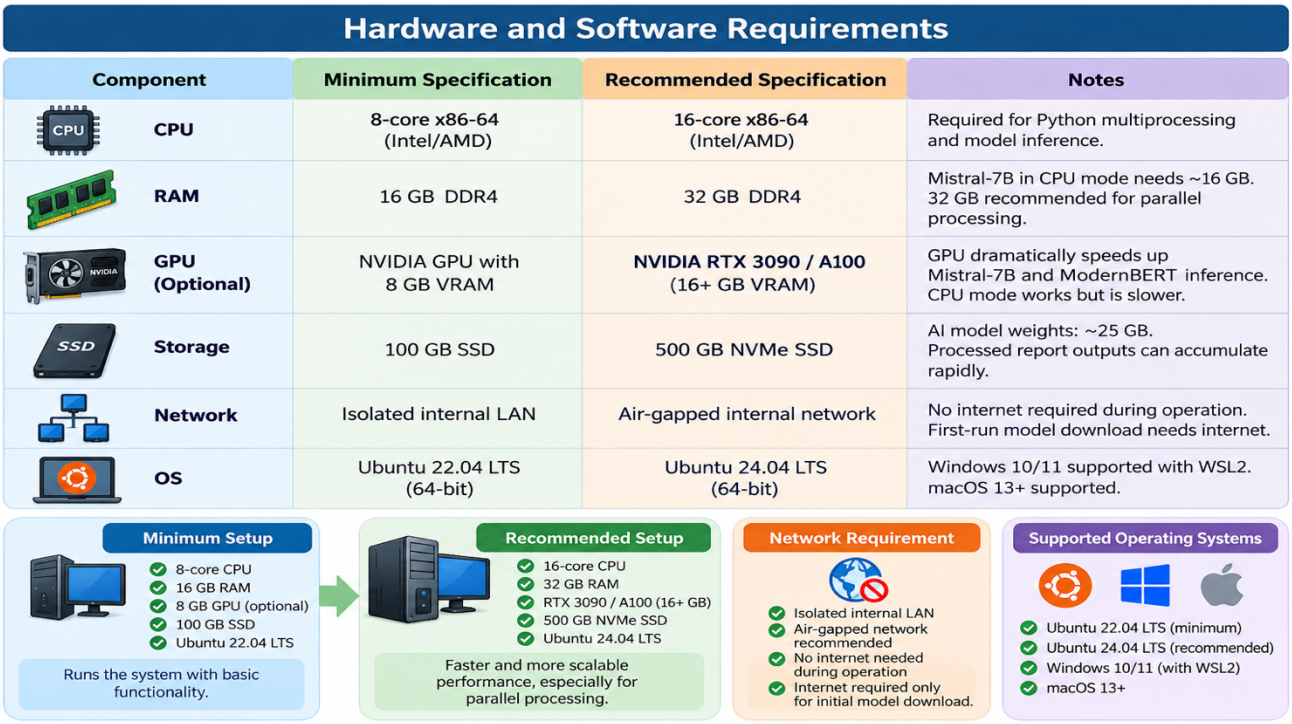
3. Hardware and Software Requirements

3.1 Minimum Hardware Requirements

This section explains the minimum and recommended computer infrastructure required to run the proposed AI system, particularly for processing documents and running models such as Mistral-7B

and ModernBERT. It specifies that the system should ideally use an 8-core CPU, at least 16 GB RAM, an optional NVIDIA GPU with 8 GB VRAM, 100 GB SSD storage, and Ubuntu 22.04 LTS as the minimum configuration. For better performance, especially when processing multiple reports or running models in parallel, it recommends a 16-core CPU, 32 GB RAM, a higher-memory GPU such as an RTX 3090/A100, and a 500 GB NVMe SSD. The system is designed to operate on an isolated internal network without requiring internet access during normal operation, although internet is needed initially to download the AI models. Overall, the table is telling us what hardware and operating environment is needed to deploy and efficiently run the AI-based document/report processing system, with the recommended specifications providing faster and more scalable performance than the minimum setup.

Component	**Minimum Specification**	**Recommended Specification**	**Notes**
CPU	8-core x86-64 (Intel/AMD)	16-core x86-64	Required for Python multiprocessing and model inference.
RAM	16 GB DDR4	32 GB DDR4	Mistral-7B in CPU mode needs ~16 GB. 32 GB recommended for parallel processing.
GPU (Optional)	NVIDIA GPU with 8 GB VRAM	NVIDIA RTX 3090 / A100 (16+ GB)	GPU dramatically speeds up Mistral-7B and ModernBERT inference. CPU mode works but is slower.
Storage	100 GB SSD	500 GB NVMe SSD	AI model weights: ~25 GB. Processed report outputs can accumulate rapidly.
Network	Isolated internal LAN	Air-gapped internal network	No internet required during operation. First-run model download needs internet.
OS	Ubuntu 22.04 LTS (64-bit)	Ubuntu 24.04 LTS (64-bit)	Windows 10/11 supported with WSL2. macOS 13+ supported.



3.2 Software Dependencies — Complete List

Package	Version	Purpose	Install Command
Python	3.10 +	Core runtime	System install or pyenv
PyMuPDF (fitz)	1.24+	PDF text + image extraction	pip install pymupdf
python-docx	1.1+	DOCX text extraction	pip install python-docx
camelot-py	0.11+	PDF table extraction	pip install camelot-py[cv]
openpyxl	3.1+	XLSX table extraction	pip install openpyxl
pytesseract	0.3.10+	Python wrapper for Tesseract OCR	pip install pytesseract
Tesseract OCR	5.x	LSTM-based OCR engine (system package)	sudo apt install tesseract-ocr
OpenCV (cv2)	4.9+	Image preprocessing for OCR	pip install opencv-python
pdf2image	1.17+	PDF-to-image for OCR pipeline	pip install pdf2image
Pillow (PIL)	10.x+	Image handling	pip install pillow
python-magic	0.4.27+	MIME type detection (magic bytes)	pip install python-magic
transformers	4.47+	ModernBERT, NER models, Mistral	pip install transformers
torch	2.2+	PyTorch — inference backend	pip install torch (CPU) or pip install torch --index-url https://download.pytorch.org/whl/cu121 (GPU)

sentence-transformers	3.x+	BGE-M3 embeddings	pip install sentence-transformers
setfit	1.x+	SetFit few-shot classifier	pip install setfit
bitsandbytes	0.43+	4-bit quantisation for Mistral-7B	pip install bitsandbytes
accelerate	0.30+	Model loading acceleration	pip install accelerate
jsonschema	4.x+	JSON schema validation	pip install jsonschema
dicttoxml	1.7+	JSON → XML conversion	pip install dicttoxml
reportlab	4.x+	PDF summary generation	pip install reportlab
ClamAV	1.3+	Offline malware scanning (system)	sudo apt install clamav
Flask	3.x+	REST API wrapper (optional)	pip install flask

Section 3.2 explains all the software libraries, frameworks, and system tools required to build and run the proposed AI-based document/report processing system. Python 3.10+ acts as the main programming environment, while PyMuPDF and python-docx extract text and images from PDF and Word documents, and Camelot and OpenPyXL extract tables from PDF and Excel files. For scanned documents, Tesseract OCR, Pytesseract, OpenCV, pdf2image, and Pillow work together to convert document images into machine-readable text: pdf2image converts PDF pages to images, OpenCV/Pillow preprocess the images, and Tesseract performs OCR. python-magic identifies the type of uploaded files so the system can select the appropriate processing pipeline.

The AI/ML part is handled mainly by Transformers and PyTorch, which provide the framework and inference engine for models such as ModernBERT and Mistral-7B. Sentence-Transformers generates semantic embeddings such as BGE-M3 embeddings, while SetFit can perform few-shot classification when only a small amount of labelled data is available. bitsandbytes enables 4-bit quantization to reduce the memory requirements of Mistral-7B, and Accelerate helps load and run models efficiently, particularly on GPUs. The remaining tools support the system's output and deployment: jsonschema validates structured JSON results, dicttoxml converts JSON into XML, ReportLab generates PDF summaries, ClamAV performs offline malware/security scanning of uploaded files, and Flask can optionally expose the whole pipeline as a REST API. In simple terms, this section defines the complete software stack—from document ingestion and OCR, through table/text extraction and AI model inference, to validation, security scanning, report generation, and API deployment.

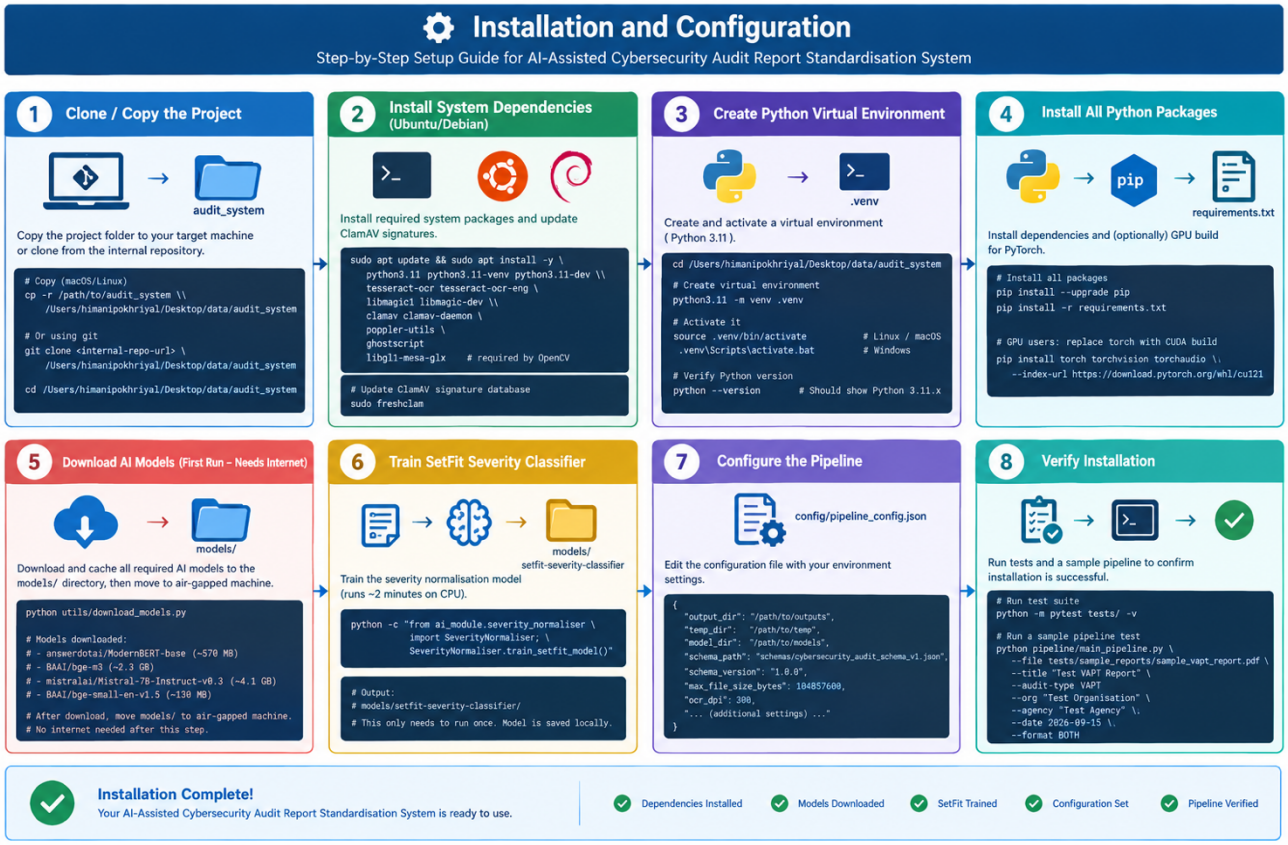
4. Installation and Configuration

4.1 Step-by-Step Installation

The installation and configuration phase establishes the software environment, system dependencies, machine-learning models, configuration parameters, and validation setup required for the AI-Assisted Cybersecurity Audit Report Standardisation System. The process begins by copying or cloning the complete project repository onto the target machine, ensuring that all application modules, configuration files, schemas, utilities, and supporting resources are available in the required project structure. The necessary operating-system dependencies are then installed to support Python execution, OCR processing, malware scanning, document processing, PDF handling, and computer-vision

operations. A dedicated Python virtual environment is created using the required Python version to isolate the project's dependencies from other software installed on the system. The required Python packages are subsequently installed within this environment, with GPU-enabled libraries optionally configured where compatible hardware is available.

The next stage prepares the artificial-intelligence models required by the system. The required models, including ModernBERT, BGE-M3, Mistral-7B-Instruct, and the model resources used by SetFit, are downloaded and stored locally. This initial model preparation requires internet connectivity; once the models have been downloaded, they can be transferred to the air-gapped environment so that normal system operation does not depend on external services. The SetFit severity classifier is then trained and stored locally for use in severity normalisation. After the AI components have been prepared, the pipeline configuration is established through the configuration file, which defines parameters such as model locations, output and temporary directories, schema version, OCR settings, confidence thresholds, model usage, processing limits, and debugging options. Finally, the complete installation is verified by executing the project's test suite and processing a representative sample audit report. Successful verification confirms that the software environment, dependencies, AI models, configuration, and processing pipeline are functioning correctly and that the system is ready for operational use. This installation sequence supports the project's offline and air-gapped architecture by ensuring that the required software, models, and supporting resources are available locally before normal processing begins.



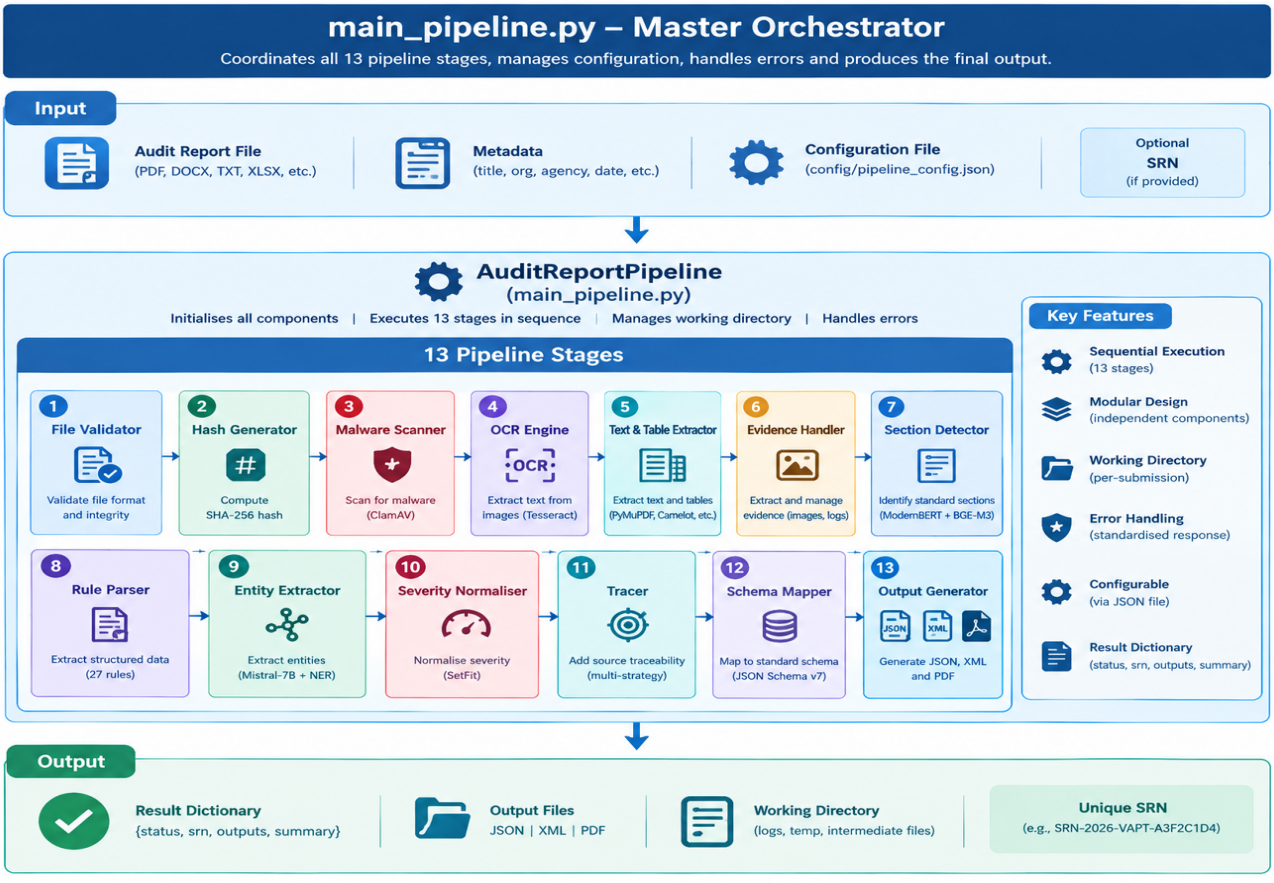
5. Module-by-Module Description

The main_given module functions as the central orchestration component of the AI-Assisted Cybersecurity Audit Report Standardisation System. Its primary responsibility is to coordinate the complete audit-report processing workflow by instantiating the required pipeline components, loading the system configuration, and executing the defined processing stages in the correct sequence. The master orchestrator acts as the central entry point for processing an audit report and receives the input

file together with relevant metadata such as the report title, organisation, agency, audit date, and submission reference number when available. It also manages the per-submission working directory, ensuring that temporary files, intermediate processing results, extracted evidence, and other working resources remain organised for each individual submission.

During processing, the master orchestrator coordinates the execution of all thirteen pipeline stages, beginning with file validation and continuing through security scanning, OCR and document extraction, evidence handling, section detection, rule parsing, entity extraction, severity normalisation, source traceability, schema mapping, and final output generation. It also computes a SHA-256 hash of the submitted file to provide a reliable digital representation of the input and supports the generation of a unique Submission Reference Number (SRN) for identifying each processed audit report. The orchestrator further provides centralised error handling so that failures occurring during processing can be represented through a standardised result structure rather than producing inconsistent responses from individual modules.

The final result generated by the master pipeline is returned as a structured result dictionary containing the processing status, Submission Reference Number, generated output locations, and a processing summary. The output may include standardised JSON, XML, and PDF representations of the processed audit report together with summary information such as the number of findings, detected sections, extracted evidence items, extracted tables, and whether OCR processing was applied. In this way, main pipeline provides the control layer that integrates the independent modules into a single sequential processing workflow and ensures that the complete cybersecurity audit report standardisation process is executed in a controlled, traceable, and consistent manner. The module therefore serves as the operational backbone of the system, connecting input handling, processing components, configuration management, error management, and final output generation within one coordinated pipeline.



6. Rule-Based Parser — Detailed Documentation

6.1 Overview

The Rule-Based Parser (RuleBasedParser class) is Stage 8 of the pipeline. It applies 27 compiled regular-expression (regex) rules to the full document text to extract predictable, structured entities. Regex rules are deterministic, transparent, and extremely fast — processing a 50-page report in under 1 second. Each rule is documented with four metadata fields: pattern, derivation (why this pattern was designed), purpose (what it extracts), and ai_handoff (what the AI module does with the extracted value).

6.2 Why Rule-Based First, Then AI?

Characteristic	Rule-Based Parser	AI/NLP Module
Speed	< 1 second per document	10–60 seconds per document
Accuracy on structured patterns	Very high (CVE, CVSS, IP — 100% if well-formed)	High but may miss exact formats
Accuracy on unstructured text	Cannot extract paraphrased content	High — handles variations
Transparency	Fully transparent — pattern is visible	Black-box — reasons are internal
Dependency	Zero ML models needed	Requires model weights
Maintenance	Edit the rules dict directly	Requires re-training or fine-tuning

> **Design Principle**
> Rules handle what is definite and patterned. AI handles what is variable and contextual. The two components are complementary: rules provide high-confidence anchors that the AI can use as context.

6.3 Complete Rule Registry — All 27 Rules

Rule Name	Pattern (Simplified)	Derivation	Purpose	AI Handoff
CVE_ID	CVE-YYYY-NNNNN	MITRE CVE format	Extract vulnerability identifiers	AI looks up CVSS score, maps to CWE
CWE_ID	CWE-NNN	MITRE CWE format	Extract weakness type IDs	AI maps CWE to category
CVSS_SCORE	CVSS: N.N (0–10)	CVSS numeric score format	Extract severity score	AI validates range, cross-checks severity label
CVSS_VECTOR	CVSS:3.1/AV:N/AC:L/...	CVSS v3.x vector string spec	Capture full CVSS vector	None — rule captures complete string

IP_V4	N.N.N.N (0–255 per octet)	IPv4 dotted-decimal	Extract affected IP addresses	AI links IP to asset name via context
IP_V6	XXXX:XXXX::XXXX	RFC 5952 IPv6 notation	Extract IPv6 addresses	None
MAC_ADDRESS	XX:XX:XX:XX:XX:XX	IEEE 802 MAC format	IoT/network device identification	AI maps OUI prefix to manufacturer
PORT_NUMBER	port NNNNN or :NNNNN	TCP/UDP port range 1–65535	Identify exposed ports	AI maps port to well-known service
URL	https?://domain/path	RFC 3986 HTTP/HTTPS URL	Extract vulnerable endpoint URLs	AI extracts path, flags sensitive routes
DOMAIN	name.tld (gov.in, nic.in...)	DNS domain with Indian-context TLDs	Identify affected domains	None
SEVERITY_LABEL	critical/high/medium/low/info	Vocabulary from CVSS, CERT-In, agencies	Extract raw severity label	SetFit normalises to 5-class standard
DATE_DMY	DD/MM/YYYY, DD-MM-YYYY	Indian standard date format	Extract report and audit dates	AI determines which date type
DATE_TEXT	15 September 2026 etc.	Text date format in narratives	Extract human-readable dates	None
MD5_HASH	32 hex characters	MD5 output length	File hashes in evidence / IOCs	None
SHA256_HASH	64 hex characters	SHA-256 output length	File integrity hashes	None
MQTT_TOPIC	mqtt:// or /sensors/topic	MQTT protocol + IoT topic naming	IoT: unencrypted MQTT topics	AI classifies topic as sensitive
IOT_PROTOCOL	MQTT/CoAP/Zigbee/BLE/LoRa/Modbus etc.	OWASP IoT Top 10	Identify IoT-	AI checks for known

		+ NIST SP 800-213	specific protocols	protocol vulnerabilities
FIRMWARE_VERSION	firmware version vX.Y.Z	Firmware version string pattern	Device firmware version extraction	AI cross-refs with CVE database
SECTION_HEADING	ALL CAPS or Title Case line	Document heading heuristic	Detect potential section headings	SectionDetector classifies to standard section
HTTP_METHOD	GET/POST/PUT/DELETE/PATCH	RFC 7231 HTTP methods	Web app: HTTP method in findings	None
HTTP_STATUS	100–599	HTTP status code ranges	Web app: response codes in evidence	AI interprets code in context
OWASP_CATEGORY	A01:2021 – Name	OWASP Top 10 2021 format	Web app: compliance mapping extraction	None
PHONE_IN	10-digit Indian numbers	TNAI numbering plan	Contact number extraction	None
EMAIL	user@domain.tld	RFC 5322 email format	Auditor/org contact extraction	None
EMPANELMENT_ID	CERT-In empanelment format	CERT-In agency ID pattern	Agency accreditation extraction	None
EXPLOIT_CODE	exploit / PoC / payload keywords	Common PoC terminology	Flag presence of exploit evidence	AI extracts full PoC text from context
NETWORK_RANGE	N.N.N.N/NN (CIDR)	CIDR notation standard	Network scope extraction	AI maps to specific asset count

6.4 Rule Parser Source Code — Key Excerpt

The Rule-Based Parser is a deterministic information-extraction component used to identify predefined cybersecurity entities from unstructured document text. It uses a fixed set of 27 pattern-matching rules, primarily based on regular expressions, to detect information such as CVE identifiers, CVSS scores, IP addresses, severity levels, URLs, and IoT protocols. The document is processed page by page, and each rule is applied to the text to identify matching entities. The patterns are compiled once during initialization to reduce repeated computational overhead and improve processing efficiency when multiple documents are analyzed.

For each detected entity, the parser retains the extracted value, page number, and surrounding textual context, thereby providing evidence and provenance for every extraction. Where applicable, the most specific capture group is selected to obtain the actual entity rather than unnecessary surrounding text. After extraction, duplicate entities are removed using case-insensitive comparison while retaining the first occurrence. Thus, the overall theoretical process can be represented as document text → rule-based pattern matching → entity extraction → context and page attribution → deduplication → structured cybersecurity information, providing a transparent and explainable preprocessing layer for subsequent cybersecurity analysis and machine-learning systems.

7. AI Component — Detailed Documentation

7.1 Overview

The AI component is distributed across three modules in the ai_module/ directory. Together they implement the intelligent, context-aware extraction capabilities that rule-based parsing alone cannot provide. The three modules are:

- section_detector: Classifies document headings to standard section names (ModernBERT + BGE-M3)
- entity_extractor: Extracts structured findings, tools, and narrative fields (Mistral-7B + ModernBERT NER)
- severity_normaliser: Normalises raw severity labels to 5-class standard (SetFit)

7.2 Module 1: Section Detector

> **Problem Being Solved**
> Different agencies use completely different section headings for the same content: 'Findings', 'Observations', 'Issues Identified', 'Security Gaps', and 'Vulnerabilities Detected' all refer to the same standard section 'Vulnerability Findings'. The Section Detector maps any heading to the correct standard section.

Three-Tier Classification Architecture

Tier	Method	When Used	Confidence Range
Tier 1	Synonym rule matching (exact + partial)	When heading exactly or partially matches known synonyms	90–100%
Tier 2	ModernBERT zero-shot/fine-tuned classifier	When synonym rules don't match; uses 8192-token context model	70–95%
Tier 3	BGE-M3 embedding cosine similarity	When ModernBERT confidence < threshold; semantic fallback	50–85%

ModernBERT — Why This Model?

ModernBERT (released December 2024 by Answer.AI) is the most recent significant BERT-family model. Key advantages over standard BERT/RoBERTa:

Feature	BERT-base (2018)	RoBERTa-base (2019)	ModernBERT-base (2024)
Max context length	512 tokens	512 tokens	8192 tokens — 16× more
Positional encoding	Absolute sinusoidal	Absolute learned	Rotary (RoPE) — better on long docs
Attention	Full global attention	Full global attention	Alternating local + global — efficient
Training data	16 GB BooksCorpus + Wiki	160 GB text	2 Trillion tokens (diverse web + code)
GLUE benchmark	84.6	88.5	90.2 (estimated)
Ideal for	Short text tasks	Short text tasks	Long documents (ideal for audit reports)

BGE-M3 — Embedding Fallback

BGE-M3 (BAAI, 2024) generates dense vector embeddings. For section heading classification, we pre-compute embeddings for all standard section names (and their synonyms) at load time. For any new heading, its embedding is computed and compared via cosine similarity.

```
# Pre-compute once at model load:
for section in STANDARD_SECTIONS:
    candidates = [section] + SYNONYM_MAP.get(section, [])
    embeddings = bge_model.encode(candidates, normalize_embeddings=True)
    section_embeddings[section] = np.mean(embeddings, axis=0)  # averaged

# At inference:
heading_emb = bge_model.encode([heading], normalize_embeddings=True)[0]
similarities = {section: np.dot(heading_emb, emb)
                for section, emb in section_embeddings.items()}
best_section = max(similarities, key=similarities.get)
confidence   = similarities[best_section]   # cosine sim = 0.0–1.0
```

7.3 Module 2: Entity Extractor

> **Problem Being Solved**
> Even after section detection, the content within each section is unstructured prose. The Entity Extractor uses Mistral-7B-Instruct (for high-quality structured JSON extraction) and ModernBERT NER (for entity tagging) to convert unstructured text into structured finding records.

Three-Tier Extraction Architecture

Tier	Method	Best For	Fallback Trigger
Tier 1	Mistral-7B-Instruct JSON extraction	Narrative text, complex paragraph findings	Primary method when LLM available
Tier 2	Table-based extraction (header matching)	Findings presented in tables	When Tier 1 returns no findings

Tier 3	ModernBERT NER (token classification)	Semi-structured bullet lists	When Tiers 1 & 2 return no findings

Mistral-7B-Instruct — Why This Model?

Criterion	Mistral-7B-Instruct	GPT-4 (OpenAI)	Llama-3.1-8B
Deployment	Fully offline / air-gap	Requires internet API	Fully offline
Cost	Free (open weights)	Paid API ($)	Free (open weights)
4-bit quantisation	Yes (GPTQ, NF4)	Not applicable	Yes
RAM in 4-bit mode	~5 GB VRAM / ~8 GB RAM	Not applicable	~6 GB VRAM
JSON instruction following	Excellent (v0.3)	Excellent	Good
Cybersecurity domain knowledge	Good	Excellent	Good
MEITY compliance	Suitable (open source)	Requires external API — problematic for air-gap	Suitable

Prompt Template for Finding Extraction

```
prompt = f"""[INST] You are a cybersecurity audit analyst.
Extract all vulnerability findings from the following {audit_type} audit
report text. Return ONLY a valid JSON array. Each finding must have:
- vulnerability_name (string)
- vulnerability_description (string)
- severity (one of: Critical, High, Medium, Low, Informational)
- affected_asset (string or null)
- affected_endpoint (string or null)
- cve_id (CVE-YYYY-NNNNN or null)
- cwe_id (CWE-NNN or null)
- cvss_score (0-10 or null)
- recommendation (string or null)

Text: {text_chunk[:4000]}

Return ONLY the JSON array. [/INST]"""
```

7.4 Module 3: Severity Normaliser

> **Problem Being Solved**
> Different agencies classify severity using different vocabularies: 'P0', 'Urgent', 'Emergency', 'Very High' all mean Critical. 'Orange' means Medium. The SetFit classifier normalises all of these to the 5-class standard without needing hundreds of labelled examples.

SetFit — Why This Model?

SetFit (Sentence Transformer Fine-tuning, 2022) is a HuggingFace framework for few-shot text classification. It works in two phases:

- Phase 1 — Contrastive fine-tuning: Pairs of same-class and different-class examples are used to fine-tune a sentence transformer so that same-class texts cluster together in embedding space.
- Phase 2 — Classification head: A simple logistic regression head is trained on the fine-tuned embeddings.

Result: With only 8 labelled examples per class (40 total for 5 classes), SetFit achieves accuracy comparable to full BERT fine-tuning on hundreds of examples.

Metric	Standard Fine-tuning (BERT)	SetFit	GPT-3 few-shot
Examples needed per class	100–1000	8–16	10–20 (in prompt)
Training time	10–60 min	2–5 min	None (inference only)
Inference speed	Fast	Fast	Slow (API call)
Offline capable	Yes	Yes	No
Accuracy (few-shot setting)	Moderate (data-hungry)	High	High (but expensive)

Three-Tier Normalisation

Tier	Method	Example
Tier 1	SetFit classifier (ML)	'P0' → Critical (predicted by model)
Tier 2	Rule-based synonym map	'urgent' matches Critical synonyms → Critical
Tier 3	CVSS score-to-severity mapping	Score '9.8' → Critical (9.0–10.0 range)

7.5 Source Traceability Module

> **Module Role**
> Every extracted field is tagged with its precise source location in the original document: source filename, page number, paragraph number, and a raw text snippet. This allows reviewers to verify any AI-extracted value against the original report.

Traceability Search Strategy

Strategy	Method	When Used
Exact text search	re.search(escaped_query, page_text)	For short, distinctive values (CVEs, IPs, headings)
Word-overlap search	Count matching keywords in page text	For AI-paraphrased content (summaries, descriptions)
Table row search	Search all table cells	For findings extracted from tables

8. JSON Schema Documentation

8.1 Schema Overview

File: schemas/cybersecurity_audit_schema_v1.json

The master schema defines the complete structure of a standardised cybersecurity audit report. It covers all four domains (VAPT, Network, Web Application, IoT) in a single unified schema, with domain-specific fields isolated in a domain_specific_fields object.

Schema Property	Value
$schema	http://json-schema.org/draft-07/schema#
Version	1.0.0
Total top-level fields	16 (14 required + 2 optional)
Total nested fields	200+
Domains covered	VAPT, Network, Web Application, Mobile, IoT
Traceability support	Yes — source_trace definition on every entity

The Cybersecurity Audit JSON Schema provides a standardised structural framework for representing cybersecurity audit reports in a consistent and machine-readable format. The master schema covers the major audit domains, including VAPT, Network, Web Application, Mobile, and IoT, while maintaining domain-specific information within a dedicated domain_specific_fields structure. The schema follows the JSON Schema Draft-07 specification and contains approximately 16 top-level fields and more than 200 nested fields. It also incorporates source traceability, enabling extracted entities and findings to be linked to their original file, page, paragraph, table, or textual evidence, thereby supporting data validation, interoperability, reproducibility, and auditability.

8.2 Top-Level Required Fields

Field	Type	Description
report_metadata	object	SRN, hash, dates, file info, correction history
audit_type	object	Primary type, sub-type, testing approach
audited_organisation	object	Name, sector, contact, location
auditing_agency	object	Name, empanelment ID, team, certifications
audit_scope	object	Description, in/out-of-scope systems, period
audit_objectives	array of strings	Stated audit objectives
audit_methodology	object	Description, frameworks, automated vs manual
assets_covered	array of objects	Each asset: type, IP, hostname, URL, criticality
tools_used	array of objects	Tool name, version, category, purpose
vulnerability_findings	array of objects	Core findings — 20+ fields per finding
recommendations	array of objects	Linked to findings, with remediation type and effort
executive_summary	object	Text, finding counts by severity, overall risk
technical_summary	object	Text, attack vectors, most-affected components
conclusion	object	Text, overall security posture rating
approval_status	object	Status, approver, timestamp, approval token

The top-level schema defines the major structural components required to represent a complete cybersecurity audit report. These include report metadata, audit type, audited organisation, auditing agency, audit scope, objectives, methodology, assets, tools, vulnerability findings, recommendations,

executive summary, technical summary, conclusion, and approval status. Each field has a predefined data type and purpose, ensuring that information such as organisation details, testing scope, security findings, remediation recommendations, risk summaries, and approval information is captured in a uniform manner. This structure enables different audit reports to follow the same standard representation while still allowing flexibility for domain-specific technical information.

8.3 Vulnerability Finding Record — Full Field List

Field	Type	Required	Description
finding_id	string	Yes	Auto: FIND-001, FIND-002, ...
vulnerability_name	string	Yes	Normalised vulnerability name
vulnerability_description	string	No	Full extracted description
severity	enum	Yes	Critical / High / Medium / Low / Informational
cvss_score	number 0–10	No	CVSS numerical score
cvss_vector	string	No	Full CVSS vector string
cvss_version	enum	No	2.0 / 3.0 / 3.1 / 4.0
cve_id	string	No	CVE-YYYY-NNNNN pattern enforced
cwe_id	string	No	CWE-NNN pattern enforced
affected_asset	string	Yes	IP / hostname / URL / device name
affected_component	string	No	Specific component or service
affected_endpoint	string	No	URL path, API endpoint, port
affected_parameter	string	No	Request parameter name
attack_vector	enum	No	Network/Adjacent/Local/Physical
proof_of_concept	string	No	PoC description or code
impact.confidentiality	enum	No	None / Low / High
impact.integrity	enum	No	None / Low / High
impact.availability	enum	No	None / Low / High
evidence	array	No	Linked evidence items (screenshots, logs)
recommendation	string	No	Remediation advice
remediation_priority	enum	No	Immediate / High / Medium / Low
compliance_mapping	array	No	Framework + control ID + control name
domain_specific_fields	object	No	VAPT / Network / Web / IoT sub-objects
extraction_confidence	number 0–1	No	AI confidence score for this finding
source_traceability	object	No	File, page, paragraph, table, snippet

The Vulnerability Finding Record defines the standard structure for representing individual security findings identified during an audit. Each finding contains core identification and classification information such as finding ID, vulnerability name, severity, CVSS score, CVE/CWE identifiers, affected assets and components, attack vector, impact, evidence, remediation recommendations, and compliance mappings. Additional fields capture extraction confidence and source traceability, allowing each finding to be associated with its supporting evidence and original document location. The use of controlled values, such as predefined severity and remediation-priority categories, together with validation constraints for identifiers and numerical scores, improves consistency, reduces ambiguity, and supports reliable downstream analysis.

8.4 Domain Extension Schemas

Domain	Schema File	Key Additional Fields

VAPT	vapt_schema.json	Kill chain stages reached, critical attack paths, retest status, privilege escalation risk, lateral movement risk
Network	network_schema.json	Network topology, IP ranges scanned, open services summary, CIS Controls v8 compliance coverage table
Web Application	web_application_schema.json	OWASP Top 10 2021 coverage table, SSL/TLS analysis, technology stack, total endpoints tested
IoT	iot_schema.json	Device inventory, firmware analysis summary (hardcoded creds, debug symbols), physical security (JTAG/UART/SPI), OWASP IoT Top 10 coverage

The Domain Extension Schemas provide specialised fields for different cybersecurity testing domains while maintaining compatibility with the common master schema. The VAPT extension captures information such as attack-chain stages, critical attack paths, privilege escalation, lateral movement, and retesting status; the Network extension represents network topology, scanned IP ranges, open services, and CIS Controls coverage; the Web Application extension includes OWASP Top 10 coverage, SSL/TLS analysis, technology-stack information, and tested endpoints; and the IoT extension covers device inventories, firmware security, hardcoded credentials, debugging interfaces, physical interfaces such as JTAG/UART/SPI, and OWASP IoT Top 10 coverage. This modular design allows each domain to retain its specialised security information without compromising the consistency of the overall audit-report structure.

9. Maintenance and Future Updates

9.1 Decoupled Report-Structure Update Module

The schema versioning system allows the standard report format to be updated without disturbing the deployed pipeline. Schema versions are stored in schemas/ with a version suffix (e.g., cybersecurity_audit_schema_v1.1.json). The pipeline reads the active schema version from pipeline_config.json.

Update Scenario	**System Impact**
Add a new optional field	No impact on existing outputs
Add a new mandatory field	All new outputs will include the field; old outputs remain valid under previous schema version
Add a new audit domain	No impact on existing domains
Update synonym vocabulary	No retraining needed; takes effect immediately
Add new regex rule	Takes effect immediately on restart
Retrain SetFit severity model	No other changes needed; model is hot-swappable

10.2 Logging and Monitoring

All pipeline stages write structured logs via utils/logger.py. Log files are written to logs/ with the submission SRN in each log line for easy filtering.

```
# Example log output:
[2026-10-01 14:32:11] [SRN-2026-VAPT-A3F2] [main_pipeline] Stage 1: File Validation
[2026-10-01 14:32:11] [SRN-2026-VAPT-A3F2] [file_validator] Validation PASSED: report.pdf (PDF, 2400.1 KB)
[2026-10-01 14:32:12] [SRN-2026-VAPT-A3F2] [main_pipeline] Stage 4: OCR Processing
[2026-10-01 14:32:15] [SRN-2026-VAPT-A3F2] [ocr_engine] OCR complete: 22 pages, applied=False
[2026-10-01 14:32:28] [SRN-2026-VAPT-A3F2] [section_detector] Section detection complete: 11 sections
[2026-10-01 14:33:10] [SRN-2026-VAPT-A3F2] [entity_extractor] LLM extracted 23 findings
[2026-10-01 14:33:12] [SRN-2026-VAPT-A3F2] [main_pipeline] === PIPELINE COMPLETE ===
```